

IGAD Innovations Hub - Technical Specification v1.1

1. Overview & Goals

Problem Statement

IGAD member states lack a unified platform for innovation management, resulting in fragmented proposal processes, inconsistent documentation, and limited knowledge sharing across the region.

Business Value

- **Operational Efficiency:** 60% reduction in proposal creation time through LLM assistance
- **Knowledge Centralization:** Single source of truth for innovation activities across 8 member states
- **Cost Optimization:** Serverless architecture reduces infrastructure costs by 40% vs traditional hosting
- **Regional Collaboration:** Enhanced cross-border innovation partnerships

Success Metrics

Metric	Target	Measurement
User Adoption	500+ active users within 6 months	Monthly active users
Proposal Completion Rate	85% of started proposals completed	Conversion funnel
Content Generation Speed	5x faster than manual creation	Time-to-completion tracking
System Availability	99.9% uptime	CloudWatch Synthetics
Cost per User	<\$10/month/user	AWS Cost Explorer

2. User Journeys & Functional Requirements

2.1 Main Page Module

User Journey: Innovation manager logs in, views dashboard with recent activities, navigates to specific modules.

Functional Requirements:

- **MP-001:** Display personalized dashboard with metrics (active proposals, recent publications, team activity)
- **MP-002:** Real-time notifications for proposal status changes, publication schedules
- **MP-003:** Quick access navigation to all modules with role-based visibility
- **MP-004:** Search across all content types with faceted filtering

2.2 Proposal Writer Module

User Journey: User creates proposal using templates, receives LLM suggestions, collaborates with team, exports final document.

Functional Requirements:

- **PW-001:** Template-based proposal creation with IGAD-specific formats
- **PW-002:** LLM-assisted content generation with context-aware suggestions
- **PW-003:** Real-time collaborative editing with conflict resolution
- **PW-004:** Version control with diff visualization and rollback capability
- **PW-005:** Export to PDF/DOCX with custom branding and formatting
- **PW-006:** Approval workflow with electronic signatures

2.3 Newsletter Generator Module

User Journey: Content manager aggregates sources, generates newsletter using LLM summarization, schedules distribution.

Functional Requirements:

- **NG-001:** Drag-and-drop newsletter builder with responsive templates
- **NG-002:** Automated content aggregation from RSS feeds, web sources, internal documents
- **NG-003:** LLM-powered summarization with tone and length controls
- **NG-004:** Multi-language support for IGAD official languages

- **NG-005:** Email distribution with analytics and engagement tracking
- **NG-006:** A/B testing for subject lines and content variations

2.4 Prompt Manager Module

User Journey: AI specialist creates prompt templates, versions them, runs A/B tests, analyzes performance.

Functional Requirements:

- **PM-001:** CRUD operations for prompt templates with rich text editor
- **PM-002:** Semantic versioning with changelog and approval gates
- **PM-003:** Tag-based categorization and search functionality
- **PM-004:** A/B testing framework with statistical significance testing
- **PM-005:** Performance analytics (response quality, latency, cost per token)
- **PM-006:** Prompt sharing and collaboration with access controls

2.5 Web Scraper Module

User Journey: Data analyst configures scraping jobs, monitors execution, processes results for analysis.

Functional Requirements:

- **WS-001:** Visual scraper configuration with CSS selector builder
- **WS-002:** Robots.txt compliance checking and allowlist management
- **WS-003:** Rate limiting with exponential backoff and circuit breakers
- **WS-004:** Scheduled execution with SQS-based job queuing
- **WS-005:** Data transformation pipelines with validation rules
- **WS-006:** Anti-abuse controls (IP rotation, user agent randomization, CAPTCHA detection)
- **WS-007:** Results storage with deduplication and change detection

2.6 Amazon Q Integration Module

User Journey: User asks natural language questions, receives contextual answers from organizational knowledge base.

Functional Requirements:

- **AQ-001:** Natural language query interface with conversation history

- **AQ-002:** Integration with all module data sources (proposals, newsletters, scraped content)
- **AQ-003:** Role-based access control for query results
- **AQ-004:** Citation tracking with source attribution
- **AQ-005:** Query analytics and knowledge gap identification
- **AQ-006:** Custom knowledge base connectors for external systems

3. Non-Functional Requirements

Category	Requirement	Target	Measurement
Performance	API Response Time (p95)	≤ 800ms	CloudWatch API Gateway metrics
Performance	LLM Response Time (p95)	≤ 2.5s	Custom CloudWatch metrics
Performance	Page Load Time (p95)	≤ 2s	Real User Monitoring
Availability	System Uptime	≥ 99.9%	CloudWatch Synthetics
Reliability	Recovery Point Objective (RPO)	≤ 15 minutes	Aurora automated backups
Reliability	Recovery Time Objective (RTO)	≤ 60 minutes	Disaster recovery testing
Scalability	Concurrent Users	1000+	Load testing with Artillery
Scalability	Auto-scaling Response	< 2 minutes	Lambda cold start metrics
Security	Authentication Success Rate	> 99%	Cognito metrics
Security	Data Encryption	100% at rest/transit	Compliance audit
Usability	Mobile Responsiveness	320px+ viewports	Automated testing
Cost	Monthly Cost per User	< \$10	AWS Cost Explorer allocation

4. Architecture Summary

Serverless-First Design

The system leverages AWS serverless services to minimize operational overhead and optimize costs. Lambda functions handle all compute workloads, with API Gateway providing HTTP endpoints and Cognito managing authentication. Aurora Serverless v2 provides managed database scaling, while S3 and CloudFront deliver static assets globally.

Data Flow

1. **Ingestion:** Web scrapers populate SQS queues, triggering Lambda processors
2. **Processing:** NestJS services orchestrate business logic, calling LLM APIs for content generation
3. **Storage:** Structured data in Aurora, documents in S3, search indices in OpenSearch
4. **Delivery:** CloudFront serves React SPA, API Gateway routes backend requests

Failure Modes & Mitigation

- **Lambda Timeout:** Circuit breakers with graceful degradation
- **Database Connection Limits:** Connection pooling with RDS Proxy
- **LLM API Failures:** Retry logic with exponential backoff, fallback to cached responses
- **Queue Overflow:** Dead letter queues with manual intervention alerts

Scaling Approach

- **Horizontal:** Lambda auto-scaling based on request volume
- **Vertical:** Aurora Serverless capacity scaling based on CPU/memory utilization
- **Geographic:** CloudFront edge locations for global content delivery

5. Detailed Component Design

5.1 Frontend Application (React)

Route Structure

/dashboard	- Main page with metrics and navigation
/proposals	- Proposal management interface
/proposals/new	- Proposal creation wizard
/proposals/:id	- Proposal editor with collaboration
/newsletters	- Newsletter management
/newsletters/new	- Newsletter builder
/prompts	- Prompt library and testing
/scraper	- Web scraping configuration
/search	- Amazon Q interface
/settings	- User preferences and admin

State Management

- **Global State:** Zustand for user session, app configuration
- **Server State:** TanStack Query for API caching and synchronization
- **Form State:** React Hook Form for complex forms with validation

Authentication Flow

```
// Cognito integration with automatic token refresh
const useAuth = () => {
  const [user, setUser] = useState<CognitoUser | null>(null);

  useEffect(() => {
    Auth.currentAuthenticatedUser()
      .then(setUser)
      .catch(() => setUser(null));
  }, []);

  return { user, signIn: Auth.signIn, signOut: Auth.signOut };
};
```

5.2 Backend Services (NestJS)

Module Architecture

```
src/
├─ auth/           - Cognito JWT validation, RBAC
├─ proposals/      - Proposal CRUD, collaboration, export
├─ newsletters/    - Newsletter generation, distribution
├─ prompts/        - Prompt management, A/B testing
├─ scraper/        - Job scheduling, result processing
├─ search/         - Amazon Q integration, query routing
├─ llm/            - LLM orchestration, guardrails
└─ common/         - Shared utilities, decorators, pipes
```

Error Contracts

```
export class ApiError {
  code: string;
  message: string;
  details?: Record<string, any>;
  timestamp: string;
  requestId: string;
}

export enum ErrorCodes {
  VALIDATION_FAILED = 'VALIDATION_FAILED',
  RESOURCE_NOT_FOUND = 'RESOURCE_NOT_FOUND',
  UNAUTHORIZED = 'UNAUTHORIZED',
  RATE_LIMIT_EXCEEDED = 'RATE_LIMIT_EXCEEDED',
  LLM_SERVICE_UNAVAILABLE = 'LLM_SERVICE_UNAVAILABLE'
}
```

DTOs with Validation

```
export class CreateProposalDto {
  @IsString()
  @Length(1, 500)
  title: string;

  @IsOptional()
  @IsUUID()
  templateId?: string;

  @IsObject()
  @ValidateNested()
  content: ProposalContent;

  @IsArray()
  @IsString({ each: true })
  tags: string[];
}
```

5.3 LLM Orchestration Patterns

Guardrails Implementation

```
@Injectable()
export class LLMGuardrailService {
  async validateInput(prompt: string): Promise<ValidationResult> {
    // Check for prompt injection patterns
    // Validate content policy compliance
    // Rate limiting per user/organization
  }

  async validateOutput(response: string): Promise<ValidationResult> {
    // Content safety scanning
    // Factual accuracy checks where applicable
    // Format validation
  }
}
```

Prompt Templates

```
export interface PromptTemplate {
  id: string;
  name: string;
  template: string;
  variables: PromptVariable[];
  guardrails: GuardrailConfig;
  version: string;
}

export class PromptExecutor {
  async execute(template: PromptTemplate, variables: Record<string, any>): Promise<LLMR
    const prompt = this.interpolateTemplate(template.template, variables);
    await this.guardrailService.validateInput(prompt);

    const response = await this.llmService.generate(prompt, template.guardrails);
    await this.guardrailService.validateOutput(response.content);

    return response;
  }
}
```

5.4 Amazon Q Integration

Connector Architecture

```
@Injectable()
export class AmazonQService {
  async query(question: string, userId: string): Promise<QResponse> {
    const userContext = await this.getUserContext(userId);
    const filteredSources = this.applyAccessControls(userContext);

    return this.qClient.query({
      question,
      dataSourceIds: filteredSources,
      userContext: {
        userId,
        groups: userContext.groups
      }
    });
  }

  private applyAccessControls(userContext: UserContext): string[] {
    // Filter data sources based on user permissions
    // Apply organizational boundaries
    // Respect data classification levels
  }
}
```

Index Scope & Permissions

- **Proposals:** User can access own + shared within organization
- **Newsletters:** Published content accessible to all users
- **Scraped Content:** Access based on source classification
- **Prompts:** Public library + private user prompts

5.5 Web Scraper Controls

Robots.txt Compliance

```
@Injectable()
export class RobotsService {
  private robotsCache = new Map<string, RobotsDirective>();

  async canScrape(url: string, userAgent: string): Promise<boolean> {
    const domain = new URL(url).hostname;
    const robots = await this.getRobotsDirective(domain);

    return robots.isAllowed(url, userAgent);
  }

  async getRobotsDirective(domain: string): Promise<RobotsDirective> {
    if (!this.robotsCache.has(domain)) {
      const robotsTxt = await this.fetchRobotsTxt(domain);
      this.robotsCache.set(domain, new RobotsDirective(robotsTxt));
    }
    return this.robotsCache.get(domain);
  }
}
```

Rate Limiting & Anti-Abuse

```
@Injectable()
export class ScraperRateLimiter {
  private rateLimits = new Map<string, TokenBucket>();

  async acquirePermit(domain: string): Promise<boolean> {
    const bucket = this.getRateLimitBucket(domain);
    return bucket.tryConsume(1);
  }

  private getRateLimitBucket(domain: string): TokenBucket {
    if (!this.rateLimits.has(domain)) {
      // Default: 1 request per second per domain
      this.rateLimits.set(domain, new TokenBucket(1, 1));
    }
    return this.rateLimits.get(domain);
  }
}
```

6. Interfaces & Contracts

REST Endpoints

Proposals API

GET	/api/v1/proposals	– List proposals with pagination
POST	/api/v1/proposals	– Create new proposal
GET	/api/v1/proposals/{id}	– Get proposal by ID
PUT	/api/v1/proposals/{id}	– Update proposal
DELETE	/api/v1/proposals/{id}	– Delete proposal
POST	/api/v1/proposals/{id}/export	– Export to PDF/DOCX
POST	/api/v1/proposals/{id}/share	– Share with collaborators

LLM Generation API

POST	/api/v1/llm/generate	– Generate content from prompt
POST	/api/v1/llm/summarize	– Summarize provided text
GET	/api/v1/llm/templates	– List available prompt templates
POST	/api/v1/llm/templates/{id}/test	– Test prompt template

Web Scraper API

GET	/api/v1/scraper/jobs	– List scraping jobs
POST	/api/v1/scraper/jobs	– Create scraping job
PUT	/api/v1/scraper/jobs/{id}	– Update job configuration
DELETE	/api/v1/scraper/jobs/{id}	– Delete scraping job
POST	/api/v1/scraper/jobs/{id}/run	– Trigger immediate execution
GET	/api/v1/scraper/results	– Query scraping results

Error Model

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Request validation failed",
    "details": {
      "field": "title",
      "constraint": "Length must be between 1 and 500 characters"
    },
    "timestamp": "2025-10-15T20:12:20.045Z",
    "requestId": "req_abc123"
  }
}
```

Idempotency Keys

All POST/PUT operations support `Idempotency-Key` header to prevent duplicate operations:

```
POST /api/v1/proposals
Idempotency-Key: prop_create_20251015_abc123
```

7. Data Model

Core Entities

Users

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  cognito_id VARCHAR(255) UNIQUE NOT NULL,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  organization_id UUID REFERENCES organizations(id),  
  role VARCHAR(50) NOT NULL,  
  preferences JSONB DEFAULT '{}',  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

Proposals

```
CREATE TABLE proposals (  
  id UUID PRIMARY KEY,  
  title VARCHAR(500) NOT NULL,  
  content JSONB NOT NULL,  
  template_id UUID REFERENCES proposal_templates(id),  
  status VARCHAR(50) DEFAULT 'draft',  
  author_id UUID REFERENCES users(id),  
  collaborators UUID[] DEFAULT '{}',  
  version INTEGER DEFAULT 1,  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP DEFAULT NOW()  
);
```

Prompts

```
CREATE TABLE prompts (  
  id UUID PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  template TEXT NOT NULL,  
  variables JSONB DEFAULT '[]',  
  category VARCHAR(100),  
  tags TEXT[] DEFAULT '{}',  
  version VARCHAR(20) NOT NULL,  
  is_active BOOLEAN DEFAULT true,  
  author_id UUID REFERENCES users(id),  
  usage_count INTEGER DEFAULT 0,  
  avg_rating DECIMAL(3,2),  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Scrape Jobs & Results

```
CREATE TABLE scrape_jobs (  
  id UUID PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  url_pattern VARCHAR(1000) NOT NULL,  
  schedule_cron VARCHAR(100),  
  config JSONB NOT NULL,  
  status VARCHAR(50) DEFAULT 'active',  
  last_run TIMESTAMP,  
  next_run TIMESTAMP,  
  created_by UUID REFERENCES users(id)  
);
```

```
CREATE TABLE scrape_results (  
  id UUID PRIMARY KEY,  
  job_id UUID REFERENCES scrape_jobs(id),  
  url VARCHAR(1000) NOT NULL,  
  content_hash VARCHAR(64),  
  data JSONB NOT NULL,  
  scraped_at TIMESTAMP DEFAULT NOW(),  
  INDEX idx_job_scraped (job_id, scraped_at)  
);
```

Data Retention & Lifecycle

Entity	Retention Period	Lifecycle Policy
Proposals	7 years	Archive to S3 Glacier after 2 years
Newsletters	3 years	Move to S3 IA after 90 days
Scrape Results	1 year	Delete after 12 months
Audit Logs	5 years	CloudWatch Logs → S3 → Glacier
User Sessions	30 days	Auto-expire Cognito tokens

8. Security & Compliance

IAM Roles & Least Privilege Matrix

Role	Aurora	S3	OpenSearch	SQS	Lambda Invoke
Lambda Execution	Read/Write specific tables	Read/Write app buckets	Index/Search	Send/Receive	-
Scraper Service	Write scrape_results	Write raw data	Index content	Receive jobs	-
API Gateway	-	-	-	-	Invoke functions
Cognito Users	-	Read own files	Search with filters	-	-

Encryption Strategy

- **S3**: SSE-S3 for general content, SSE-KMS for sensitive documents
- **Aurora**: Encryption at rest with AWS managed keys
- **OpenSearch**: Encryption at rest and in transit
- **Lambda**: Environment variables encrypted with KMS
- **API Gateway**: TLS 1.2+ enforced, WAF with OWASP rules

PII Handling

- **Data Classification:** Automatic PII detection in scraped content
- **Anonymization:** Hash/tokenize personal identifiers before storage
- **Access Logging:** All PII access logged with user attribution
- **Retention:** PII purged according to data protection regulations

Threat Model & Mitigations

Threat	Impact	Mitigation
Prompt Injection	High	Input validation, output filtering, rate limiting
Scraping Abuse	Medium	Robots.txt compliance, rate limiting, IP allowlists
Data Exfiltration	High	IAM least privilege, VPC endpoints, audit logging
DDoS	Medium	CloudFront + WAF, API Gateway throttling
Credential Compromise	High	MFA enforcement, session timeout, anomaly detection

9. Deployment & Environments

Environment Strategy

- **Test Environment:** Separate AWS account, reduced capacity, auto-pause Aurora
- **Production Environment:** Dedicated AWS account, multi-AZ deployment, full monitoring

SAM Packaging

```
# template.yaml
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31

Parameters:
  Environment:
    Type: String
    AllowedValues: [test, prod]

Mappings:
  EnvironmentConfig:
    test:
      AuroraMinCapacity: 0.5
      AuroraMaxCapacity: 2
      LambdaMemory: 512
    prod:
      AuroraMinCapacity: 2
      AuroraMaxCapacity: 16
      LambdaMemory: 1024

Resources:
  ApiGateway:
    Type: AWS::Serverless::HttpApi
    Properties:
      StageName: !Ref Environment
      Auth:
        DefaultAuthorizer: CognitoAuthorizer
        Authorizers:
          CognitoAuthorizer:
            JwtConfiguration:
              issuer: !Sub "https://cognito-idp.${AWS::Region}.amazonaws.com/${UserPool}"
              audience:
                - !Ref UserPoolClient
```

Rollback Strategy

1. **Blue/Green Deployment:** SAM CodeDeploy integration with automatic rollback on CloudWatch alarms
2. **Database Migrations:** Backward-compatible changes only, separate rollback scripts

3. **Static Assets:** CloudFront invalidation with previous version fallback

10. Observability & SRE Runbooks

Dashboards

- **Application Dashboard:** API latency, error rates, user activity
- **Infrastructure Dashboard:** Lambda metrics, Aurora performance, S3 usage
- **Business Dashboard:** Proposal completion rates, newsletter engagement, scraping success

Critical Alarms

```
HighErrorRate:
  MetricName: 4XXError
  Threshold: 5
  ComparisonOperator: GreaterThanThreshold
  EvaluationPeriods: 2
  Actions:
    - !Ref PagerDutyTopic
```

```
DatabaseConnectionFailure:
  MetricName: DatabaseConnections
  Threshold: 80
  ComparisonOperator: GreaterThanThreshold
  EvaluationPeriods: 1
  Actions:
    - !Ref SlackAlertTopic
```

Incident Response Runbook

1. **Detection:** CloudWatch alarm triggers PagerDuty
2. **Assessment:** Check application dashboard, identify affected components
3. **Mitigation:** Scale Aurora capacity, restart Lambda functions if needed
4. **Communication:** Update status page, notify stakeholders
5. **Resolution:** Apply fixes, verify metrics return to normal
6. **Post-mortem:** Document root cause, update runbooks

Rollback Checklist

- ☐ Verify rollback target version
- ☐ Check database migration compatibility
- ☐ Execute SAM deployment rollback
- ☐ Validate API endpoints respond correctly
- ☐ Confirm user authentication works
- ☐ Monitor error rates for 15 minutes
- ☐ Update status page and notifications

11. Cost-Aware Architecture

Service Optimization Decisions

- **HTTP API vs REST API:** 70% cost reduction for API Gateway
- **Lambda Memory Sizing:** Right-sized at 1024MB based on profiling
- **Reserved Concurrency:** Set at 100 to prevent runaway costs
- **CloudWatch Logs:** 14-day retention, structured logging only for errors
- **CloudFront:** Long TTL (24h) for static assets, compression enabled
- **OpenSearch vs pgvector:** Evaluate based on search volume; pgvector for <10GB data

Cost Controls

```
# Lambda reserved concurrency
ReservedConcurrency: 100

# Aurora Serverless scaling
ServerlessV2ScalingConfiguration:
  MinCapacity: 0.5
  MaxCapacity: 16

# S3 lifecycle rules
LifecycleConfiguration:
  Rules:
    - Status: Enabled
      Transitions:
        - Days: 30
          StorageClass: STANDARD_IA
        - Days: 90
          StorageClass: GLACIER
```

Monthly Cost Estimates

Service	Test Environment	Production
Lambda	\$50	\$300
Aurora Serverless	\$100	\$800
API Gateway	\$20	\$150
S3 + CloudFront	\$30	\$200
OpenSearch	\$200	\$1200
Total	\$400	\$2650

12. Risks, Assumptions, Decisions

High-Risk Items

- **LLM API Costs:** Potential for unexpected usage spikes

- *Mitigation*: Rate limiting, budget alerts, usage quotas per user
- **Aurora Cold Starts**: Serverless pause/resume latency
 - *Mitigation*: Keep-alive Lambda, connection pooling with RDS Proxy
- **OpenSearch Costs**: Search volume difficult to predict
 - *Mitigation*: Start with pgvector, migrate to OpenSearch if needed

Key Assumptions

- IGAD users primarily English-speaking (localization Phase 2)
- Web scraping volume <1M pages/month
- LLM responses average 500 tokens
- Peak concurrent users <200 during business hours

Architecture Decisions

Decision	Rationale	Trade-offs
NestJS on Lambda	Familiar framework, good TypeScript support	Cold start latency vs development speed
Aurora Serverless v2	Auto-scaling, cost-effective for variable load	Pause latency vs operational simplicity
HTTP API Gateway	70% cost savings vs REST API	Fewer features vs cost optimization
Cognito for Auth	Managed service, integrates with API Gateway	Vendor lock-in vs reduced complexity

13. Open Questions

Question	Owner	Due Date	Impact
Multi-language support requirements?	Product Team	2025-11-01	Frontend complexity
Integration with existing IGAD systems?	IT Team	2025-10-30	API design

Question	Owner	Due Date	Impact
Data residency requirements per member state?	Legal Team	2025-11-15	Multi-region deployment
LLM model selection (GPT-4 vs Claude vs local)?	AI Team	2025-11-01	Cost and performance
Approval workflow complexity for proposals?	Business Team	2025-10-25	Backend design

14. Appendices

Appendix A: OpenAPI Excerpt

```
openapi: 3.0.3
info:
  title: IGAD Innovations Hub API
  version: 1.1.0

paths:
  /api/v1/proposals:
    post:
      summary: Create proposal
      security:
        - CognitoAuth: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateProposalRequest'
      responses:
        '201':
          description: Proposal created
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ProposalResponse'
        '400':
          description: Validation error
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'

components:
  schemas:
    CreateProposalRequest:
      type: object
      required: [title, content]
      properties:
        title:
```

```
    type: string
    minLength: 1
    maxLength: 500
  content:
    type: object
  templateId:
    type: string
    format: uuid
  tags:
    type: array
    items:
      type: string
```

Appendix B: IAM Policy Examples

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "rds-data:BatchExecuteStatement",
        "rds-data:BeginTransaction",
        "rds-data:CommitTransaction",
        "rds-data:ExecuteStatement"
      ],
      "Resource": "arn:aws:rds:*:*:cluster:igad-hub-*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::igad-hub-documents/*"
    }
  ]
}
```

Appendix C: SLO/SLI Table

SLI	SLO	Error Budget	Measurement Window
API Availability	99.9%	43.2 minutes/month	30 days
API Latency (p95)	<800ms	5% of requests	7 days
LLM Response Time (p95)	<2.5s	5% of requests	7 days
Proposal Save Success	99.95%	0.05% failure rate	24 hours
Newsletter Delivery	99.5%	0.5% failure rate	24 hours
Scraper Job Success	95%	5% failure rate	7 days